TRIA VOLUMINA · UNA LEX · EVIDENTIA AUT SILENTIUM

BUILD · TEST · PROVE · DOCUMENT · SHIP

10,000,000 AI ENGINEERING

IV

Volume IV The Curriculum

THIRTEEN PHASES FROM AI CODING TO AUTONOMOUS ENGINEERING
GROUNDED IN THE SHIPPED SYSTEM

VOLUMEN IV · QUARTA · 3,583 VERBA · ZERO DEPENDENTIAE

AEOS

EDITIO PRINCEPS · AEOS PRESS · MMXXVI ◆ OPERA QUARTA

HOW TO READ THIS CODEX ◆ VOLUMEN IV

THIS IS NOT A GUIDE. THIS IS A RECEIPT.

A guide tells you what exists. A codex tells you what was built, in what order, with what proof, and how you can verify every claim with your own hands. Nothing in these pages is a proposal — every mechanism ships in the aeos repository, every property ends in the name of the test that enforces it, and the whole system runs offline with zero runtime dependencies.

Read it as an operator: run the proofs, then read the law.

```
pip install -e .        # zero runtime dependencies
python -m pytest        # 426 proofs, chaos storm included
aeos run-demo           # the whole OS, end to end
aeos storm              # the nuclear receipt: 9/9 scenarios survive
```

INDEX OPERUM

CONTENTS

Volume IV — The Curriculum

The founding specification closed with a command: after the first version, audit everything, and keep building. The system answered with thirty-seven versions of receipts. What the specification demanded that the journey had not yet produced was the curriculum: thirteen phases, each carrying objectives, theory, practice, projects, evaluation, capabilities unlocked, and an advanced project. This volume is that curriculum — and it is not written from imagination. Every phase is grounded in artifacts that exist, commands that run, and tests that pass in the repository this book documents. Where a phase reaches past what is built, it says so plainly: the ladder's last rungs are named, not pretended. The curriculum's law is the project's law: **build before explain, receipt before claim.**

PHASE 1 — AI CODING

The floor of the discipline: a model is a component with a typed seam, never a person and never an oracle.

Objectives — Treat every model call as an infrastructure boundary: versioned, metered, swappable, and failure-typed. Learn that "prompt" is an interface, and interfaces deserve contracts.

Theory — Model agnosticism is not a courtesy; it is economics and survival. Providers change price, availability, and quality weekly. The system that hard-binds to one provider inherits its outages. The seam pattern: a caller speaks in typed envelopes, an adapter translates to the wire, a taxonomy names every failure mode that can return. In AEOS the seam is `models.py` (the slot abstraction), `adapters.py` (the taxonomy-preserving translations), and `providers.py` (OpenRouter, Abacus, OpenAI — behind one gate, live keys opt-in, spend dollar-capped).

Practice — Run `aeos live-check`: it resolves the live configuration and spends nothing. Run `aeos run-demo`: the reference loop completes offline — proof that the model is an accelerator, not a dependency. Then `aeos run-demo --live` with a provider key and watch the same loop, metered, with the budget cutoff armed.

Projects — (1) Wrap one provider behind a typed adapter that maps its wire errors onto your failure taxonomy. (2) Add a second provider and prove the caller cannot tell which one answered.

Evaluation — `test_v11_providers.py` proves live mode refuses to run without explicit keys and never logs them; the taxonomy tests prove every adapter error is named, never a raw exception. The metric that matters: **zero provider-specific imports in the calling code** — machine-checked by the doctor's zero-dependency scan.

Capabilities unlocked — Swappable intelligence. Everything built in later phases composes through this seam.

Advanced project — A routing policy that chooses a provider per task by cost and latency, with the choice recorded in the evidence bundle — the triangle (Phase 12) is this project grown up.

PHASE 2 — AGENTIC CODING

A loop, not a call: plan, act, observe, gate, repeat — with every step leaving evidence.

Objectives — Turn the model call into a loop whose steps are checkable, whose side effects are bounded, and whose claim of success must survive a gate that does not trust it.

Theory — An agent is a loop with state and tools; reliability comes from what surrounds the loop, not from the loop's confidence. The core law: **no envelope, no result** — every step returns a typed envelope; every completion claim is checked against declared gates. The reference pipeline (`pipeline.py`) is seven tasks from intent to evidence bundle; the harness (`harness.py`) is the execution environment; `sandbox_runner.py` isolates untrusted work in a subprocess killed at its wall clock.

Practice — `aeos up` (the front door) runs the entire loop: preflight, workspace, work, shutdown — a numbered receipt at the end. `aeos resume` then crashes the loop deliberately and completes it with side effects executed exactly once.

Projects — (1) Add a tool to the loop behind a write-boundary declaration; prove a rogue write is reverted. (2) Give the loop a poisoned input and require a written verdict instead of a crash.

Evaluation — `test_harness.py` and `test_e2e.py` drive the loop end-to-end; the poisoned-input and wall-clock-kill cases are tests, not anecdotes. `test_v17_resume.py` proves crash-resume with side-effects-once.

Capabilities unlocked — Durable execution. The loop can now be killed, audited, and resumed — the precondition for everything long-running later.

Advanced project — A loop that proposes its own next task and must pass the standards gate (Phase 4) to execute it — the seed of Phase 11.

PHASE 3 — CONTEXT ENGINEERING

The unit of cost is not tokens spent; it is tokens spent per outcome — and most of them were never needed.

Objectives — Feed work the minimum sufficient context, in layers, with retrieval paying for itself.

Theory — Context is a budget, not a bucket. The recall ladder — keys, then snippets, then full records — spends the smallest representation that answers, and falls back only on need. Cache telemetry turns the cache from superstition into arithmetic: hit rate, effective tokens, savings per bundle. The `context_os.py` assembly is explicit and inspectable; nothing enters a prompt by accident.

Practice — `aeos recall --query "deploy research"` walks the ladder live. `aeos telemetry` reports the cache hit rate and effective token count. `aeos dividend` shows distillation paying rent — negative marginal token consumption when memory does its job.

Projects — (1) Instrument a real workload and plot effective tokens per outcome across ten runs. (2) Add a distillation layer and prove the marginal cost of the eleventh run is lower than the first.

Evaluation — `test_v15_recall.py` proves each layer returns what it promises; `test_v22_telemetry.py` proves the accounting arithmetic; `test_v14_dividend.py` proves the rent law — memory that does not pay is evicted.

Capabilities unlocked — Sublinear cost. The system's context bill grows slower than its experience.

Advanced project — A context policy that learns per class of task which layers suffice, with the policy itself versioned and reversible.

PHASE 4 — SKILLS ENGINEERING

Knowledge that lives in prose decays; knowledge that lives as checkable standards refuses to.

Objectives — Encode operator law and craft as named, versioned units that plans must cite — and that the system can refuse to proceed without.

Theory — A skill is a contract between intent and execution: preconditions, steps, evidence. The standards pattern makes law load-bearing: operator rules are registered as `[STD-n]`; a plan that cites no standard is refused before it starts. This inverts the usual failure — instead of discovering mid-flight that requirements were folklore, the absence of cited law stops the work at the gate.

Practice — `aeos standards --init` writes the STANDARDS.md template; `aeos standards` audits a plan against the register. `skills.py` carries the skill contracts the factory later validates.

Projects — (1) Write the three standards your last incident proved you needed, and wire them to refuse the plan that skipped them. (2) Convert one prose runbook into a cited standard and delete the prose.

Evaluation — `test_v19_standards.py` proves uncited plans are refused and cited plans proceed; the charter check (Phase 13) proves cited tests exist in the suite — law that cannot silently rot.

Capabilities unlocked — Plans that carry their law. The next phase's agents inherit rules, not vibes.

Advanced project — A standards linter that proposes new `[STD-n]` entries from failure post-mortems, gated by the learning system (Phase 11).

PHASE 5 — AGENT ENGINEERING

An agent is a loop plus identity, authority, and a record — and a tendency to misreport that must be designed against.

Objectives — Build agents as bounded workers whose artifacts are verified from the filesystem, never self-reported; whose authority is granted, scoped, and revocable; whose failure modes are named.

Theory — Three laws carry this phase. **Verification from the fs-diff, not the mouth:** a companion that claims success is believed only when the filesystem shows the work (phantom refusals in `companions.py`). **Authority is a token, not a mood:** sponsorship is scoped, one-shot, and expires (`sponsor`). **A boundary means revert:** a rogue agent's writes outside its declared boundary are rolled back. Fleet state is an append-only event stream — observability as replayable proof, not as a promise.

Practice — `aeos companions` shows the Pi/aider/Claude-style nodes and their enable path. `aeos fleet` runs CRUD over the fleet with the live event stream. `aeos sponsor --scope ...` issues a token; `aeos console` renders the authority record.

Projects — (1) Add a companion and prove its phantom report is refused while its real work is accepted. (2) Issue a scoped token and prove the out-of-scope action fails with the token in hand.

Evaluation — `test_v12_companions.py` and `test_v20_companions2.py` cover fs-diff verification, boundary revert, wall-clock kills; `test_v16_fleet.py` replays the stream and proves it append-only.

Capabilities unlocked — Delegation you can audit. Teams (Phase 8) become safe to compose.

Advanced project — An agent whose authority decays with observed reliability — the governor (Phase 11) applied per-agent.

PHASE 6 — HARNESS ENGINEERING

The harness is the product: everything that makes imperfect agents safe to run.

Objectives — Design the execution environment so that power cuts, kill storms, full disks, and hostile inputs are receipts, not disasters.

Theory — The harness laws: every persistent write is atomic (tmp, fsync, rename); every load is tolerant (torn lines quarantine, never crash); locks are kernel-released (a killed run cannot strand a workspace); offline is provable (a full run with sockets disabled). Chaos is a drill with a receipt: nine scenarios — SIGKILL storms, torn files, disk-full, garbage inputs, memory caps, socket blackout — run inside the test suite, in CI, on every push. Backups are deterministic and restores fail closed: a corrupt backup restores nothing, never something wrong.

Practice — `aeos storm` is the nuclear receipt. `aeos vault` shows the fault-tolerance posture. `aeos backup` then `aeos restore` round-trips a workspace; tamper with the tar and watch the restore refuse.

Projects — (1) Add a tenth storm scenario for a failure your environment actually had. (2) Break atomicity on purpose in a branch and prove the suite fails.

Evaluation — `test_v26_vault.py`, `test_v27_storm.py` (all nine scenarios as tests), `test_v29_soak.py` (drilled backup, sustained operation).

Capabilities unlocked — Survivability. Long-running work (Phase 9) now has a floor to stand on.

Advanced project — A chaos scenario generator that proposes faults from the boot ledger's real failure history — the system designing its own immune exposure.

PHASE 7 — EVALUATION ENGINEERING

If you cannot grade it, you cannot improve it; if a model grades it, you have moved the problem, not solved it.

Objectives — Make evaluation mechanical: predicate judges, weights, thresholds — and the system grading its own laws.

Theory — Judges are predicates, not models: a check either holds against disk or it does not. Evaluation suites declare what is measured and the pass bar; entropy coverage measures whether the bench actually exercises the space. The self-eval pattern — the system grading its own charter — turns principle into regression test. Performance is a budget, not a boast: the envelope declares law limits and the bench measures against them at 10k scale.

Practice — `aeos eval` runs the self-eval over the charter's laws. `aeos bench --full` measures the performance envelope against its budgets. `aeos triangle` prints the control/cost/speed stance of the last run — the trade, receipted.

Projects — (1) Write the eval suite for your own operating principles, with weights and thresholds. (2) Add an entropy coverage check that fails when two cases test the same thing.

Evaluation — `test_v23_evals.py` (predicate judges), `test_v34_gauge.py` (budgets are law), `test_v13_triangle.py`.

Capabilities unlocked — Ground truth you own. Improvement (Phase 11) becomes measurable instead of narrative.

Advanced project — An adversarial eval channel: a generator whose only job is producing inputs the suite currently passes but should not — red-team as a standing role.

PHASE 8 — MULTI-AGENT ORCHESTRATION

More agents multiply coordination failures faster than they multiply output; the graph must be explicit and cycles must block.

Objectives — Orchestrate work as an explicit graph with requires-and-conditions semantics, where failure blocks dependents and cycles are refused — never hung.

Theory — Implicit orchestration (agents chatting until done) is unauditable and unbounded. The colony pattern: a plan is a declared graph; the runtime executes it; a cycle is a build error at plan time, not a hang at 3 a.m. Cross-organization composition is a trust problem before it is a protocol problem: **import is quarantine** — foreign units are untrusted material until revalidated in a local sandbox, provenance carried, sponsored install required even with a valid token in hand.

Practice — `aeos colony` runs the graph orchestration demo — including the cycle that is refused. `aeos federation-demo` walks quarantine → revalidation → sponsored install, refusing the unsponsored path with the token visibly present.

Projects — (1) Express a real three-agent workflow as a colony graph and prove the failure of one node blocks its dependents. (2) Import a foreign unit and prove the quarantine path refuses what the revalidated path accepts.

Evaluation — `test_v25_colony.py` (cycles BLOCK, failures block), `test_v9_v10.py` (federation law), `test_v21_mcp.py` (untrusted MCP imports quarantined).

Capabilities unlocked — Composition without contagion. The organization (Phase 13) can now trade capabilities safely.

Advanced project — A market where colonies bid on subgraphs with cost and reliability disclosures — federation with economics.

PHASE 9 — LONG-RUNNING AGENTS

An agent that cannot survive a weekend is a demo.

Objectives — Make duration a non-event: durable plans, crash-resume with side effects once, retention as law, and a sustained-operation receipt.

Theory — Long-running means state outlives processes. Checkpoints are atomic and versioned; schema versions fail closed on the future (state written by a newer build refuses with a remedy, never corrupts); retention archives instead of deleting; the soak receipt proves N runs on one workspace with the evidence intact. The boot ledger (Phase 10's front door) extends this to the system itself: every boot writes a receipt, including the ones that died.

Practice — `aeos resume` crashes and completes. `aeos groom --keep-runs N` archives history. `aeos soak --runs 5` produces the sustained-operation receipt; `--live` (opt-in, capped) soaks a real provider.

Projects — (1) Kill a soak at random points and prove every resume completes with side effects exactly once. (2) Add a schema-version bump drill: old state must either upgrade in place or refuse with instructions.

Evaluation — `test_v17_resume.py`, `test_v28_shipyard.py` (the v27→v33 upgrade drill is a test), `test_v29_soak.py`.

Capabilities unlocked — Duration. Autonomy (Phase 11) now has the time dimension it requires.

Advanced project — A soak that runs a week and files its own incident report — postmortem as a scheduled artifact, not a surprise.

PHASE 10 — AGENT-NATIVE SOFTWARE

Software designed to be written, tested, and operated by agents is a different shape: versioned capabilities, sponsored installation, and a front door that speaks plain language.

Objectives — Treat capabilities as products in a catalog: proposed, sandbox-validated, sponsored, installed — and make the system's own entry point operable by a tired human at 3 a.m.

Theory — The factory loop is the pattern: measure → design → sandbox-validate → propose → sponsored install. No token, no install — every refusal logged. Capability identity is content-hashed; provenance travels. And the operator surface obeys the init-system lesson: one command, four stages, exit codes scripts can branch on, every failure naming what happened and what to do — never a traceback. The boot ledger tells the next boot how the last one died.

Practice — `aeos factory-demo` (proposals only), then `aeos factory-demo --token S` (the scoped install; the second candidate refused on scope mismatch). `aeos up` with a future-schema workspace shows the plain-language failure path, exit code 2.

Projects — (1) Propose a capability, watch it validate in the sandbox, and install it with a token scoped to exactly it. (2) Write the remedy text for three failures your own environment produced — then make the system emit them.

Evaluation — `test_v5_v6_v7.py` (factory law: no token, no install), `test_v37_ignition.py` (the plain-language law: every FAIL carries a remedy; key values never enter receipts).

Capabilities unlocked — A system that can safely grow. Phase 12's OS is this property at full scale.

Advanced project — A capability marketplace with federated provenance and reliability disclosures — Phase 8's market, realized.

PHASE 11 — AUTONOMOUS ENGINEERING

Autonomy is not granted; it is earned by measured reliability, and it is spent under ceilings.

Objectives — Build the governor: autonomy levels that move only on evidence, self-improvement behind hard floors and human tokens, and leverage measured per run.

Theory — The autonomy ladder climbs on reliability evidence — `governor.py` earns levels; it never assumes them. The meta-loop (`meta.py`) may propose retirements and tunings, but hard floors are immutable and every self-modification spends a human-issued token. Leverage — outcomes per human intervention — is the number the whole discipline optimizes; the rubric's twelve leverage points are auditable against disk, not narrated. Memory that does not pay rent is evicted; failure never becomes folklore (the regression book turns it into a gate).

Practice — `aeos leverage-audit` runs the 12-point rubric against the workspace. `aeos dividend` shows the rent law. The governor's level appears in every evidence bundle with the reliability that earned it.

Projects — (1) Define the reliability evidence your ladder will accept, and make the level computed, not stored. (2) Wire one self-improvement proposal through sponsorship and prove the floor refuses the version that would regress it.

Evaluation — `test_governor.py`, `test_v18_leverage.py` (evidence on disk or it did not happen), `test_v6` meta-loop floors.

Capabilities unlocked — Trustworthy self-extension. Phase 12's OS can improve without drifting.

Advanced project — A governor that publishes its own certified reliability report per release — the notary applied to autonomy itself.

PHASE 12 — AI CAPABILITY OS

The destination of the internal ladder: the engineering system as an operating system — boot, health, ledger, proof, and law.

Objectives — Integrate every prior phase into one machine: a system that boots through stages, audits its own claims, notarizes its completions, and keeps documentation truthful by force.

Theory — An OS is judged at boot and trusted by its receipts. The ignition's four stages make startup a protocol; the doctor's PASS/WARN/FAIL rows make health a diagnosis ("a doctor that flatters is not a doctor"); the scribe makes documentation machine-checked against live reality (drift fails with file:line); the notary makes "done" a Merkle-rooted certificate, not a sentence. The outbox extends the OS to the metered edge: local WAL buffering, idempotent replay, and a wire that opens only to an explicitly named endpoint. Capabilities are permanent — every verb that ever shipped still parses, machine-checked across all tags.

Practice — `aeos doctor` (self-audit), `aeos scribe` (documentation truth), `aeos save-proof` (the certificate), `aeos outbox status` (the edge queue), `aeos up --save-proof` (a boot that notarizes its own tree).

Projects — (1) Add a doctor row for a claim your system makes about itself, and make it FAIL when the claim is false. (2) Verify a release from a cold clone: checkout, `aeos save-proof --verify … --against-tree` — the tag either matches its certificate or it does not.

Evaluation — `test_v32_doctor.py`, `test_v35_scribe.py`, `test_v36_notary.py`, `test_v37_ignition.py` — and the shipping ritual itself: notarize, tag, cold-clone verify, publish.

Capabilities unlocked — Self-governance. What remains is scale-outward, not capability-inward.

Advanced project — A second OS instance, federated with the first (Phase 8), sharing capability catalogs with provenance — the enterprise rung's foundation.

PHASE 13 — AUTONOMOUS ENTERPRISE

The frontier, honestly marked: what the system beneath the enterprise rung has proven, and what the rung itself still demands.

Objectives — State the gap precisely. The specification's last ladder rungs — autonomous business workflow and the AI-native enterprise prototype — are not built; everything beneath them is.

Theory — An enterprise is federated autonomy under a constitution. The pieces AEOS has proven: cross-org federation with quarantine (Phase 8), economics with budgets and dividends (Phase 3), a charter that is machine-checked load-bearing law (this project's PRINCIPLES.md — 40 principles, each with mechanism and test), and an OS that can grow safely (Phase 12). The pieces the rung still demands: business-workflow ontologies, long-horizon financial accountability, and multi-tenant governance at organizational scale. Naming the gap is this phase's discipline: the ladder is not climbed by relabeling.

Practice — What can be run today: `aeos federation-demo` (org-to-org trust), `aeos eval` (the constitution graded), `aeos up` (the OS under a workload). What cannot: an autonomous business workflow — and the audit (docs/SPEC-AUDIT.md) says so in writing.

Projects — (1) The honest inventory: audit your own system against its founding specification, requirement by requirement, verdicts included — this volume exists because that audit ran. (2) Charter a single business workflow as a colony graph with budgets and a human sponsor at every spend ceiling.

Evaluation — The audit's own standard: every requirement carries a verdict and an artifact; gaps are listed, never polished. Re-audit at every major version.

Capabilities unlocked — None claimed. This phase's deliverable is the frontier, drawn accurately.

Advanced project — The first business workflow run end-to-end under the constitution: sponsored, budgeted, evaluated, notarized — ladder rung eleven, earned the way rungs one through ten were.

◆

APPENDIX — THE LADDER, MAPPED

Rungs one through ten are shipped and tested (the mapping lives in docs/SPEC-AUDIT.md, requirement 53). Rungs eleven and twelve are named gaps with their foundations proven. The specification's final command — build the entire thing now — is therefore not finished, and that is the correct state for a living system to be in: the next rung is always the honest answer to "what did the last one make possible?"

The curriculum is the map. The repository is the territory. The tests are the border between them.

◆ END OF VOLUME IV · 3,583 WORDS · GENERATED FROM THE AEOS REPOSITORY ◆